

ML Experiment Triage: Debug Report

What went wrong, what caused it, and how each was resolved

Olajide Badejo

5 August 2026

Why this document exists

The main report says what the tool does and how well it works. This one says what it took to get there, which is the part that is normally thrown away and is usually the part with the most in it.

Nine entries follow. Three are defects in my code, two are cases where my test was wrong and the code was right, one is a flaw in my own measurement method, one is a toolchain misconfiguration that silently disabled the driver I had specified, one is a bug caught by reading before it could produce a wrong answer, and one is an environment decision. They are in the order they were hit.

The one worth reading if you read only one is the third: a statistical test that was running at better than twice its nominal error rate while every mechanics test around it passed.

1 Environment

No Python 3.13 on the machine

Symptom: `py -Op` listed only 3.14, where the build targets 3.13.

Root cause Nothing broken. The machine simply had a newer interpreter than the project targets.

Options Build against 3.14 and see what breaks; install 3.13; fall back to 3.12.

Fix and why Installed 3.13.14 with `py install 3.13`. Building against 3.14 would have meant betting on the wheel situation for tensorboard and protobuf, the two dependencies here most likely to lag a new interpreter, and any failure would then have been ambiguous between my code and a missing wheel.

Verification A TensorBoard write and read round trip was run before any parser code existed, so that a later parser failure could not be blamed on the toolchain.

2 Tooling

The dash guard flagged a file that is not in the repository

Symptom: the first run failed on an em dash in the build specification sitting in the working directory.

Root cause The guard walked the filesystem. The file it flagged is ignored and is an input to the work rather than part of the project, so filesystem scope was answering the wrong question. The same bug would later have flagged anything inside the virtual environment or a build directory.

Options Add path exclusions and keep adding them forever; or scope the guard to what git would ship.

Fix and why The guard now asks `git ls-files --cached --others --exclude-standard`, which is exactly the set of files that are or will become part of the repository. Exclusion lists rot; asking git does not.

Verification Passes on a clean tree, still fails on a planted dash in a tracked file, and runs twice in `make all`, once before the PDFs exist and once after, so the compiled PDFs are checked too.

texify was never running

Symptom: `make report` produced a correct PDF, but the log showed `--batch-mode: unknown option` followed by a fallback to a manual `pdflatex` sequence.

Root cause MiKTeX `texify` spells the flag `--batch`. `--batch-mode` is `pdflatex`'s spelling, and I had written the `pdflatex` one.

Two further lessons hide in this entry, both about writing the entry itself. Typing those flag names as a literal pair of hyphens in LaTeX prose typesets an en dash, and the project's dash guard caught it in the source *and* in the compiled PDF, which is the guard doing exactly the job it was built for on the document describing the guard. The obvious repair, wrapping them in `\verb`, then failed to compile at all, because `\verb` is illegal inside a macro argument and these sit inside one. The idiom that works in both places is `--{}-`, which breaks the ligature without needing verbatim mode.

Why it mattered The fallback path existed for continuous integration, where `texify` does not exist, and it worked, so the build was green and the specified driver was silently never used. A fallback that quietly becomes the primary path is worse than no fallback, because it hides the fact that the configuration is wrong.

Fix and why Corrected the flag. The fallback stays, because the Ubuntu runner genuinely needs it, but it now announces itself when it triggers.

Verification The build log names the driver it used, and now shows `texify`.

3 Statistics

The window block mode was running at 12 percent type I error

Symptom: with a nominal alpha of 0.05 and a true effect of exactly zero, the single run mode rejected the null on 12.0 percent of cases, against a gate of [2, 8] percent.

The seed replicated mode passed on its first run at 4.6 percent, so the defect was specific to the block machinery. Every mechanics test around it passed. This was found only because the calibration suite exists.

Investigation A diagnostic printed block count, block length and autocorrelation time per case, then swept type I error against window length, autocorrelation level, block length multiplier, and whether the statistic was smoothed. Two causes fell out of that grid.

Root cause, part one The autocorrelation time was estimated on the two windows *concatenated*. Joining two independent runs end to end puts a step change at the join, and the estimator reads a step change as long range dependence. Measured, it inflated the estimate by roughly a factor of two. It was also masked: the inflated value was then clipped by a target block count, so the net effect was neither reliably conservative nor reliably wrong, which is the worst kind of bug to have in a statistic.

Root cause, part two The block length was one autocorrelation time. At one τ the block means are still visibly correlated with their neighbours. The observed split is contiguous, one run's blocks against the other's, but a permuted split scatters neighbouring blocks across both groups, breaking that correlation and producing a null distribution narrower than the truth. A null that is too narrow is precisely how a test becomes anticonservative.

Smoothing made this worse in a way I had not anticipated. The nine point moving average the statistic uses raises the measured autocorrelation time of the window from about 8 to about 14, so the blocks needed to be longer than the raw series would have suggested.

Options (a) Keep one τ and accept the error rate. Rejected: the whole claim of the project is that these p values mean something. (b) Raise the multiplier and clip the block length when the window is too short. Rejected after measuring it: clipping reintroduces exactly the failure, and the measured rate at short windows stayed above 10 percent. (c) Raise the multiplier and refuse when the window cannot support it. (d) Abandon the single run mode. Rejected: sometimes one run is all there is, and a labelled weak answer beats no answer.

Fix and why Option (c). τ is now estimated per window with the larger taken, the block is three times τ , and below eight blocks the comparison raises an error naming the concrete remedies rather than returning a number. Three was chosen by measurement: across window lengths from 1200 to 8000 steps and autocorrelation coefficients from 0 to 0.9, one τ gave 9 to 14 percent, two gave 7 to 10, and three landed on nominal wherever the precondition was comfortably satisfied.

Refusing rather than clipping is the part I would defend hardest. A quietly miscalibrated p value is worse than no p value, because it spends credibility that nothing later can recover.

Verification Type I error 5.81 percent over 1928 null cases, 72 refused as too short. Power 100 percent on a large effect, confirming the fix did not simply make the test deaf. A separate test pins the refusal.

Commit Phase 2.

A selection effect in my own measurement

Symptom: after the fix above, the measured type I error still looked high in some cells: 20 percent at 3000 steps with an autocorrelation coefficient of 0.9, against 5.6 percent at 8000 steps.

Root cause Not the estimator. The refusal rule decides using τ estimated from the same

data it then tests. In the cells where most cases were refused, the survivors were exactly those whose τ happened to be underestimated, which are exactly the cases that get blocks that are too short. Conditioning on survival selected for the failure. Every cell with a refusal rate under 10 percent measured between 4.4 and 5.8 percent, and the error rate rose monotonically with the refusal rate, which is the signature of a selection effect rather than a broken statistic.

Options Refuse using something independent of the data, which for an unknown autocorrelation time is not available; report the conditional rate and say so; or run the suite inside the region where the precondition is comfortably satisfied.

Fix and why The last, with the second written down. The suite runs at 8000 steps, where under 4 percent of cases are refused, and it asserts that the refusal rate stays below 10 percent so the measurement cannot silently drift into the biased regime. The honest statement, which is in the methodology, is that the measured rate is conditional on the precondition being met.

Verification The suite asserts both the error rate and the refusal rate.

4 Reproducibility

Every HTML report differed from the last

Symptom: a test comparing two renderings from the same database at the same seed failed immediately on being written.

Root cause Plotly assigns each figure a freshly generated identifier when rendering to HTML. Nothing about the data changed; the container changed.

Why it mattered more than it looked The design says reports are pure functions of the database plus a recorded seed, and the README says so. A file that differs on every run makes that claim unverifiable: nobody can diff two reports to confirm nothing moved, which is the practical way anyone would ever check it.

Options Strip the generated identifiers after rendering, which is fragile against the library's own internal references to them; or set them explicitly.

Fix and why Set explicitly from the metric name, which is already unique per figure and is more readable in the output.

Verification Two renderings now agree line for line, ignoring only the generation timestamp, which is the one thing that legitimately differs.

String hashing would have broken reproducibility silently

Symptom: none. Caught by reading, before it could produce a wrong result.

Root cause The synthetic sweep seeded each run's generator with `hash(condition.name)`. Python randomises string hashing per process unless `PYTHONHASHSEED` is set, so the sweep would have differed on every invocation while looking entirely deterministic in the code.

Why it would have been nasty Every test in the repository would still have passed, because they all regenerate the sweep within a single process. The failure would have surfaced

only as the committed verdict record never matching, and the cause would have been very hard to see from that symptom.

Fix and why `zlib.crc32` of the name, which is stable across processes and versions. Keying on the name rather than an index also means adding a condition later cannot shift the numbers of the conditions before it.

Verification `make verify-demo` rebuilds the sweep from its seeds and checks all twelve verdicts against the committed record to four decimal places.

5 Where the test was wrong and the code was right

Recorded because being wrong in this direction is worth noticing too. In both cases the repair was to assert the real property, not to loosen the assertion.

The smoothing bleed

Symptom: a test built a step function, 900 zeros then 100 fives, and asserted the window statistic was 5.0 within 0.05. It measured 4.9444.

Root cause The nine point moving average reaches back across the step, so the four points at the start of the window each borrow some of the value before it. The deficit is $4 \times (4 + 3 + 2 + 1) / 9$ spread over 100 points, which is exactly 0.0556. The code was right; my tolerance was wrong.

Fix and why The test now asserts the derived value to six decimal places, and a second test covers the clean case where nothing bleeds. That is a stronger check than the loose one it replaced.

The self containment check

Symptom: a test asserting that no external URL appears in the HTML report failed.

Root cause The inlined Plotly bundle carries a default topojson host and several map tile attributions as string literals, in code paths a line chart never touches. None of it is fetched.

Fix and why The test now strips the inlined scripts and checks the remaining markup for tags that actually cause a network request. A test that fails on correct code is not a strict test, it is a broken one.

I was wrong about the learning rate

Symptom: a test asserting that learning rate is the strongest correlate of the validation loss failed. Batch size correlated more strongly.

Root cause Not a bug. The sweep gives learning rate an optimum in the middle of its range: 0.0003 is worse than 0.001, 0.003 is the best of the four, and 0.01 is by a distance the worst. That is a U shape, and Spearman rank correlation measures monotone association, so the correlation of a U shape is near zero however large the effect. Batch size was swept monotonically over the same runs, so it correlates more strongly while driving a smaller effect.

Options (a) Change the demo sweep so learning rate is monotone and the test passes. Rejected, and it is worth saying why: it would have made the demo flatter the method by hiding the exact case where the method is blind, and a learning rate with an optimum is the realistic shape, not the awkward one. (b) Add a nonlinear dependence measure. Deferred as scope, recorded as future work. (c) Keep the behaviour, document the limitation, pin it with a test.

Fix and why Option (c). The limitation is now in the module docstring, the methodology, the README, the main report and the caption under the sensitivity table in the HTML. The test was renamed for the blindness it checks, so that a future reader who sees a weak learning rate correlation does not “fix” it. A second assertion confirms the effect the correlation cannot see is genuinely there and correctly sized: the tool recovers a gap of 0.193 between the best and worst conditions against a designed 0.190.

Verification Two tests, one at unit level on an isolated U shape and one at integration level on the real sweep.

6 What I take from this

Three things.

The calibration suite paid for the whole project in one run. It found a statistical test running at better than twice its nominal error rate, in code that looked correct and passed every mechanics test written for it. No amount of code review would have found that number. Roughly a minute of measurement did.

Refusing is a feature. Two of the fixes above ended in the tool declining to answer: below eight blocks it raises rather than returning a p value, and below the attainable alpha it reports inconclusive rather than no change. Both were more work than the alternative and both are the reason the error rates hold.

A failing test is a question, not an instruction. Three of the nine entries here are cases where the test was wrong. In each, the temptation was to adjust the assertion until it passed, and in each the right move was to work out what the code was actually doing and then assert that precisely. One of them turned a bug report into a documented limitation of the method, which is more useful to a reader than the clean result I had expected would have been.